Deep Implicit Layers for Smooth Implied Volatility Surface Interpolation*

Rafael Zimmer
Institute of Mathematical Science and Computing
University of São Paulo - Brazil
rafael.zimmer@usp.br

***Abstract*—In this paper, we implement a neural network architecture based on deep implicit layers with the objective of optimizing the parameters of multiple SABR (stochastic alpha, beta, rho) models. The parameter values for the models are updated daily and used for generating smooth implied volatility surfaces in the context of financial options. The motivation behind our approach is to enable smooth interpolation and extrapolation of the surface, i.e., making it infinitely differentiable at all points while still maintaining some level of interpretability within our model when compared to state-of-the-art deep transformer architectures. The main contribution of the proposed architecture is to solve the problem of selecting the candidates α, ρ, ν to reduce statistical arbitrage opportunities for the respective generated surface. Our methodology includes using historical observed values of listed options as well as the calculated implied volatility for these contracts. The innate ability of transformer architectures to process observations of sequences with non-fixed lengths, i.e., for contracts with varying expiration dates is used to choose candidate points that best replicate the daily surfaces. Using our approach, the resulting surface curvature and volatility values are then compared to baseline classical methods such as linear interpolations, splines and non-implicit transformer networks.**

***Index Terms*—Neural Networks, Implied Volatility, Transformers, Quantitative Methods, Implicit Layers.**

I. Introduction

A. A Brief Introduction to Implied Volatility and Surface Reconstruction

Option contracts are one of the most important types of derivative contracts, and implied volatility surfaces play a critical role in the context of finding the correct no-arbitrage price [**?**] for these contracts. A derivative is an asset traded with respect to another asset (also called the underlying asset). An option specifically allows the buyer of the option to buy or sell (also called call and put contracts) a specified asset from the seller at a predefined future date. The pricing equations of these derivatives called the Black-Scholes model depend on the time to maturity of the contract, the price of the underlying asset to be traded, the price of the asset to be traded and the risk-free rate [**?**]. There is also an additional variable for contract pricing, called the implied volatility (*IV* or σ_{iv}). This value refers to the volatility considered, or to be expected, for the asset price at the time of maturity, and is an essential variable in the Black-Scholes equation for pricing option contracts.

Identify applicable funding agency here. If none, delete this.

Having access to the true implied volatility values allows investors to make sales at the correct prices consistent with the current no-arbitrage price of the contracts within the market. However, it is not always possible to obtain the true *IV* values that fully reflect the characteristics of existing offers. Ensuring a way to obtain this value with confidence and readily available data in the market is a crucial task for quantitative analysts [**?**]. Traditional strategies for daily estimation of these values are predominantly based on linear or spline interpolation algorithms [**?**] or stochastic differential equation models [**?**] that use numerical methods to approximate their values from existing option contracts in the market. Both methods try to approximate the correct value for the implied volatility for each pair of time to maturity and future asset price currently listed (also called strike price). This approach allows visualizing the model results in the form of a surface, where the **XY** axes are the maturity and strikes pairs and the **Z** axis is the *IVOL*. This surface is called the **implied volatility surface** (IVS). The surface, noted as $\mathcal{S}$ is therefore a function of the maturity time t and the strike price K,

$$\mathcal{S} = f(t, K), \text{ where } f : \mathbb{R}^2 \to \mathbb{R} \tag{1}$$

Ensuring accurate predictions of the IVS is crucial, as even small errors can lead to incorrect pricing and consequently large losses of profit or even negative return[**?**] given the leveraging capabilities of options when compared to non-derivative securities. Classic models such as linear interpolations, polynomial fitting, and *splines* generate arbitrage opportunities in the resulting IVS and often struggle to effectively capture market dynamics, besides lacking flexibility for extrapolation [**?**].

B. Contributions of the Presented Work

In this paper, we propose an approach to address the challenges of interpolation and extrapolation of the IVS, reducing arbitrage opportunities. The chosen method uses a stochastic differential equation solution approach, specifically the *Stochastic Alpha, Beta, Rho* (*SABR*) model. Historical values of observed implied volatilities are then used to optimize the weights of a nonlinear deep neural network.

The parameters α, ρ, ν of the *SABR* model are obtained through a simple implementation of the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton non-linear optimization algorithm [**????**]. This step is commonly performed using

only a subset of the points observed daily [**?**], due to the fact that using all points introduces various outliers that do not match the real prices of the derivative, resulting in arbitrage opportunities in the final model. To solve this difficulty, parameters are often manually filtered according to criteria set by human experts [**?**], and a subset of candidate points is used for parameter optimization. Rather than manually selecting these candidate points, the use of models capable of receiving point clouds and returning a score to select the best points automatically has been proposed by [**?**] instead. The main contribution of this research will be the use of *neural implicit layers*, a variation of fully connected linear layers that solve a fixed point equation within the final transformer architecture. This type of neural layers has been used efficiently for representing smooth densities and state-of-the-art performance when compared to plain *transformer* architectures [**??**]. Unlike regular linear neural layers, the chosen architecture can reduce the size and complexity of the final model *Kolter2020* while still being more suitable for handling unordered point clouds over time [**?**] when compared to recurrent neural layers. The optimization and filtering steps are discussed in more detail under the development section.

C. The Transformer Architecture

The *SABR* model is a stochastic volatility model used to obtain smooth surfaces, i.e., continuous and infinitely differentiable at all points [**?**]. In the context of this work, a score will be assigned for each candidate point in the daily listed options contracts. This score, called the *summarized suitability* score (*SS* score) corresponds to the suitability of the candidate point to summarize the characteristics of the generated surface, i.e., how feasible it is to reduce the number of candidates when optimizing the values for one of the parameters of the SABR model without altering the resulting surface shape or its smoothness[1]. In summary, this score value is used to remove points with the lowest score, that may introduce arbitrage in the optimization process of the *SABR* models.

Usually, the choice of subsets and the scoring process is done manually by analysts, as mentioned earlier, and the next section will focus on the details covering the *SABR* model and how we use a implicit neural layer with attention, specifically an **encoder** architecture for assigning scores automatically given the daily points candidate set. This approach is an implicit fixed-point solver variation for the scoring model proposed in [**?**]

D. Deep Implicit Layers

Differently from fully connected layers in which the transformation from input to output is defined explicitly, implicit layers are instead defined by conditions that the output must satisfy. For explicit layers, a given output z is computed directly from the input x using a function f, such that $z = f(x)$. In contrast, an implicit layer is the function g so that it depends on both the input x and the expected output z, where z is a solution for the equation and the solution to the layer, namely the weights $\mathbf{W}$ are such that

$$g_{\mathbf{W}}(x, z) = 0. \tag{2}$$

Formulating the problem as a fixed point equation can be used for a multitude of problems, including finding fixed points in recurrent networks, solutions to differential equations leading to Neural ODEs, and several optimization problems [**?**]. The implicit formulation offers several practical advantages, primarily the separation of the solution method from the layer definition which can be useful in maintaining interpretability of the resulting model[**?**].

Moreover, implicit layers offer significant benefits specifically in deep learning and automatic differentiation frameworks. Automatic differentiation methods store the entire computation graph, which can be memory-intensive. With implicit layers, however, the implicit function theorem allows computing gradients directly at the solution point and therefore storing intermediate variables becomes redundant [**?**]. This improves memory efficiency, making implicit layers particularly advantageous when substituting existing layers within large deep learning models [**?**].

II. Implementation of a Parametetric SABR Model and Implicit Neural Network Architectures

A. Defining the SABR Model Equations

The SABR model is a stochastic differential model, defined by the following equations in which the dynamics of the forward price and the corresponding volatility levels are used to calculate the implied volatility.

- **Alpha** (α): This is the initial volatility level.
- **Beta** (β): The elasticity parameter between the volatility of the asset and its price.
- **Rho** (ρ): Statistical correlation between the asset price and its volatility.
- **Volvol** (ν): The volatility of the volatility process itself.

The implied volatility function of the SABR model is then given by:

$$\sigma_{BS}(f, K) = \alpha \frac{(fK)^{\frac{1-\beta}{2}}}{1 + \left(\frac{(1-\beta)^2\rho^2}{24} + \frac{(1-\beta)^2(2-3\rho^2)}{1920}\right)(fK)^{1-\beta}} \tag{3}$$

In the context of this paper, multiple SABR models are fitted daily, corresponding to each unique maturity value at the respective day. A continuous function is used to map the different maturities to the three model parameters (values for β are fixed and remain unchanged). The three variables used to encode the maturity levels are the following:

- **P**: Given a function $\alpha_t = f_\alpha(t, \mathbf{P}) \to \mathbb{R}$, the values for α for each individual
- *SABR* model are encoded by P, such that $\mathbf{P} \in \mathbb{R}^5$.
- **Q**: Given a function $\rho_t = f_\rho(t, \mathbf{Q}) \to \mathbb{R}$, the values for ρ for each individual
- *SABR* model are encoded by Q, such that $\mathbf{Q} \in \mathbb{R}^4$.

[1]The use of a per-point score is similar to what the principal components of a collection of points is within the Principal component analysis technique

- **R**: Given a function $\nu_t = f_\nu(t, \mathbf{R}) \to \mathbb{R}$, the values for ν for each individual
- *SABR* model are encoded by R, such that $\mathbf{R} \in \mathbb{R}^4$.

And finally, the daily parametric SABR model is given by the set of encoded parameters $\{P, Q, R\}$ and the following equations to calculate values for f_α, f_ρ, e f_ν at each unique maturity value t:

$$f_\alpha(t, \mathbf{P}) = p_0 + \frac{p_3}{p_4}\left(\frac{1 - e^{-p_4 t}}{p_4 t}\right) + \frac{p_1}{p_2}e^{-p_2 t} \tag{4}$$

$$f_\rho(t, \mathbf{Q}) = q_0 + q_1 t + q_2 e^{-q_3 t} \tag{5}$$

$$f_\nu(t, \mathbf{R}) = r_0 + r_1 t^{r_2} e^{r_3 t} \tag{6}$$

The task of tuning the SABR model parameters is a supervised regression task. In machine learning, regression refers to problems where the output is a continuous variable, as opposed to classification, where the output is a discrete category. In our case, the output of the transformer model are the continuous SS scores for each parameter α, ρ, and ν. Then, the top $p = 80\%$ candidate points are passed as inputs to the BFGS algorithm, targeting the values **P, Q**, and **R** that minimize possible arbitrage opportunities of contracts with interpolated or extrapolated maturities and strikes.

B. Proposed Encoder Architecture and Error Functions

Transformers are a specific type of neural network architecture capable of capturing complex dependencies in sequential non-fixed length data. Within our proposed architecture, we use a transformer encoder to approximate the *SS* scores for each candidate point $\mathcal{S}_i^t \in \mathcal{S}^t$ from the daily observed input. Then, for each training epoch it estimates a loss score for the mean squared error between the encoder's output and a z-score obtained by Monte Carlo estimation from a candidate set $\mathcal{S}^t$ of $k = 1e2$ generated subsets from the original set. This is done to calculate the estimated z-score of the point, which is finally used as the target value for our model. Consequentially, if after training the model has converged to a mean low error for the estimated scores, the network parameters are deemed optimized to substitute the Monte Carlo process in estimating daily SS scores for the currently listed option contracts [**?**].

```
[caption={The function $f$ calculates, for each maturity, the parameter's value using
SS_Score(maturities, candidate_set, p, k, sigma)
    optimal_parameter = OptimizeForParameter(maturities)

    for candidate_point in candidate_set
        // Generate k subsets of size p
        // that contain the candidate point
        subset <- GenerateSubset(candidate_set, p, k)
        suboptimal_parameter <- OptimizeForParameter(subset)

        error <- Mean(AbsoluteDifference(
            f(maturities, suboptimal_parameter)
            ))

        ss_score <- Sqrt(e^(-Error / sigma))

        Append ss_score to subset_scores

    z_score <- (ss_score - Mean(ss_score)) / S

    return z_score
```

C. Details on the Choosen Encoder Architecture

Unlike recurrent neural networks (RNNs) and convolutional neural networks (CNNs), transformers do not rely on a sequential structure to process data. Instead, they use an attention mechanism, which allows each input element (or token) to consider the relationship with all other elements in the same sequence simultaneously. This is achieved through a series of layers called encoders (for encoding the input data) and decoders (for generating the output).

We specifically use the self-attention mechanism, which accepts continuous data and calculates attention for different heads through value (`value`), key (`key`), and query (`query`) matrices, resulting in an output weighted by the importance of each part of the sequence [**??**]. When dealing with self-attention, the three matrices are the same.

$$\textit{Attention}(Q, K, V) = \textit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{7}$$

Transformers are classified as machine learning algorithms due to their ability to learn patterns and relationships from data. They are trained using supervised techniques. In the context of the specific architecture we implemented, we use only the encoder block, with 4 input dimensions, 8 attention heads per block, and 8 blocks in total. The hidden values are passed to fully connected (FC) layers, and a final implicit fixed-point iteration block is used to obtain the scores as *SS* z-normalized values. The model training stage is carried out on historical option data listed for the **S&P 500** from January to December of 2021. The training epochs adjust the layer weights with an error function that minimizes the difference between its predictions and the Monte Carlo expected z-score

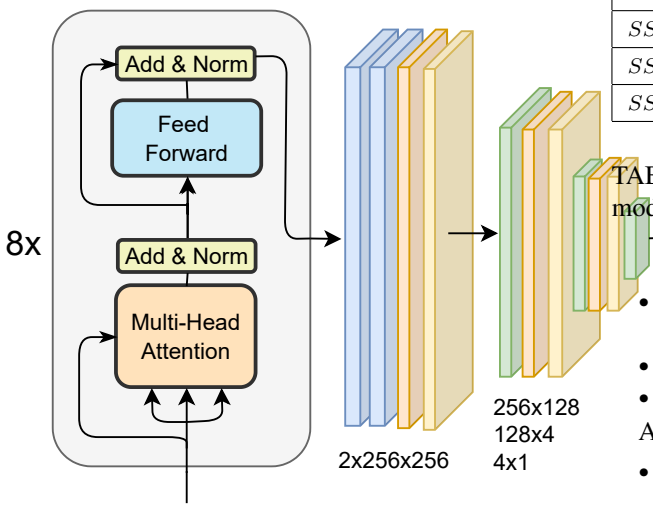

Fig. 1: Visualization of the Transformer Encoder and Implicit Fixed Point Iteration Layers

1) Details on the Implicit Layer: As defined in the previous chapter, we choose the implicit layers as the final hidden layers for our architecture for its summarization capabilities. We use the Anderson Acceleration framework [**?**] as suggested by [**?**] for NLP and other sequential models. We chose the Deep Equilibrium Model variation for the Implicit Layer, which means using the following fixed point equation instead of the one at Eq. 2

$$z^* = f(z^*, x) \tag{8}$$

The batched non-normalized Jacobian to be used as the gradient function in our code is given by the following equation, where $z*$ is the target output and f is our activation function. Finally, y are the hidden values as well as the final output normalized scores.

$$\left(\frac{\partial z^*(\cdot)}{\partial(\cdot)}\right)^T y = \left(\frac{\partial f(z^*, x)}{\partial(\cdot)}\right)^T g. \tag{9}$$

III. Methodology and Results

A. Data Collection and Pre-Processing

The data used in this work were obtained from historical options data on the SPY asset from the US stock exchange, from 2021 to 2022, available on Kaggle. This dataset contains various columns about traded options, indexed by date, including strike prices, future prices, maturities, implied volatilities, among others []. The file can be and was downloaded and stored locally in a specific directory for processing.

B. Dataset Description

The dataset contains the following columns relevant to our study, in addition to several others:

- **underlying**: Price of the underlying asset.
- **strike**: Option's strike price.

Model	Test Loss (RMSE)	Train time (Transformer)	Preprocess Time (Monte Car
SST_α	1.21	∼4m21s	∼7m50s
SST_ρ	0.72	∼2m5s	∼6m34s
SST_ν	0.75	∼1m43s	∼6m43s

TABLE I: Model training metrics, for each of the three SST models.

- **daysToExpiration**: Days remaining until the option's expiration.
- **iv**: Value for the implied volatility.
- **dt**: Current date, used as the preprocessed set index.

Additionally, we added two calculated columns:

- **maturity**: Calculated by dividing *daysToExpiration* by 252 (working days in a year) to normalize the maturity time.
- **r**: Risk-free rate obtained from the Federal Reserve Economic Data (FRED) for the corresponding period.
- **d**: Dividend yield paid quarterly by the SPY.

C. Data Preprocessing

The data preprocessing involved several essential steps, predominantly performed to allow the use of the data directly in the transformer architecture and subsequent use to generate the surface for historical data. The transformer takes four dimensions as input, which are:

1) Time to maturity;
2) The value of the parameter (α, ρ, or ν);
3) **1** if the parameter was calculated for the current day, **0** if it was calculated for a fixed set of maturities (top 7 most frequent maturities) using the previous day fitted parameters;
4) Value calculated using the previous day's $\mathbf{P}$, $\mathbf{Q}$ and $\mathbf{R}$ parameters for same strike and maturity.

The target variable is the scores generated using all candidate points for the current day, and the model's error function uses a percentage **p** of points and is trained to minimize the quadratic error function of the scores.

D. Results and Surface Comparison

We used a linear interpolation of the daily observed points as the baseline and compared it with the model's performance concerning internal points to a grid generated for points within the daily range, both for maturity values and strike.

Our transformer model was trained on the dataset, with 68 observations, each with sequences averaging 700 maturity and strike combinations. With an early stopping of $\epsilon = 10^{-3}$ sensitivity, the training was performed with a maximum epoch range of 10^2 epochs.

IV. Conclusion

A. Summary

This work presents an architecture taking as inspiration approaches from both [**?**] and [**?**] for the task of adjusting

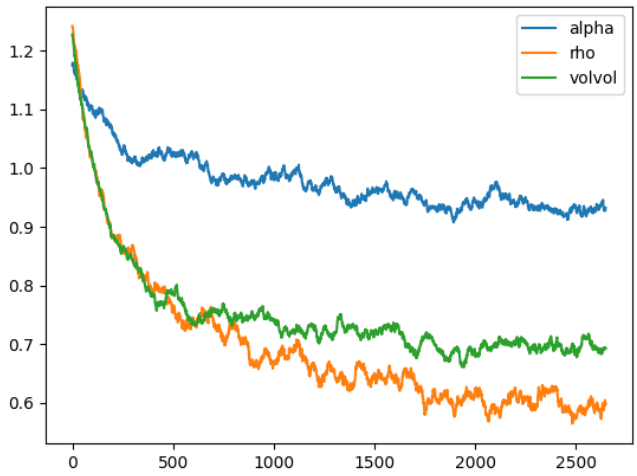

Fig. 2: Historical loss function from a Transformer training session on train data.

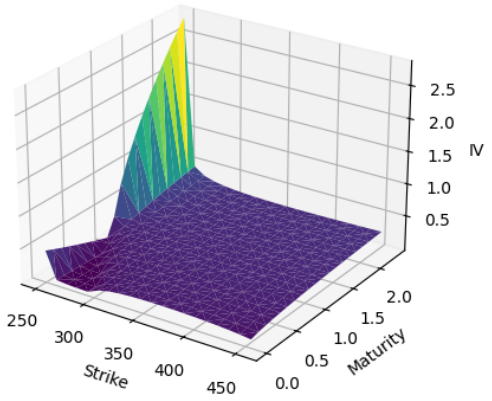

Fig. 3: Surface generated using a manually optimized SABR model. Notice how the non-removal of possible outlier candidates generates a out-of-range peak in the surface.

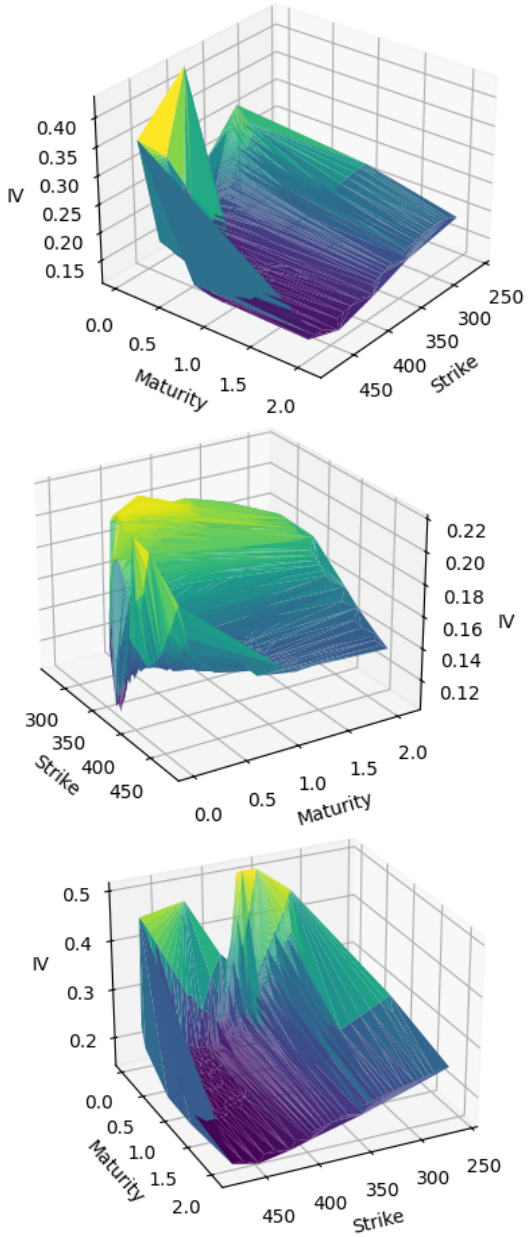

Fig. 4: Surfaces interpolated on test data using a linear interpolation model.

the parameters of the SABR model using a neural network based on the transformer architecture (specifically the *encoder* element) with implicit deep equilibrium layers as the regression component. The combination of the SABR model with the transformer architecture was used to replicate the patterns of the daily implied volatility surface, generating a smooth surface that reduces arbitrage opportunities for the existing contracts observed on the day. The candidate scoring and selection process is performed automatically, instead of manually, and the fine-tuning of the model to new daily observations is extremely fast as shown by comparing the Monte Carlo estimation method to the Model's training time in

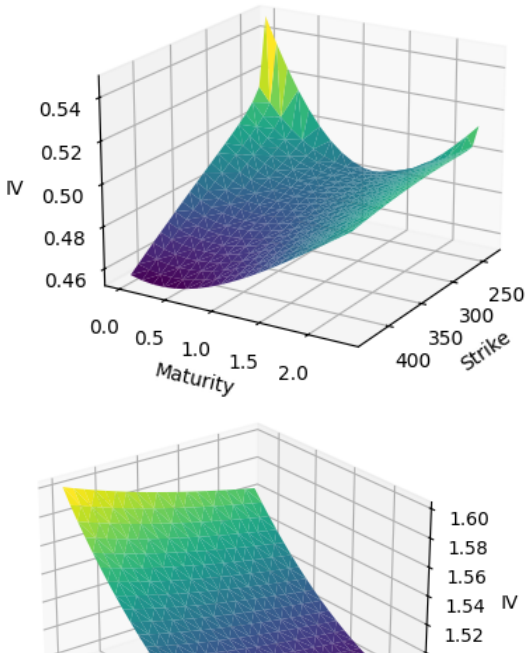

Table I. The possibility of using these models on modern GPU, which allows for faster model training times allow investors to fine-tune models and identify new trading oportunities in much smaller time intervals.

B. Implications and Future Work

The main focus of this work was to merge the approach using the transformer and the implicit layers to replace the manual selection of scores for candidates, as well to show the possibility of future research focusing on High-Frequency Trading (HFT) strategies options arbitrage strategies. Additional market factors and input features are also an interesting improvement to the chosen approach. Furthermore, it is important to consider the choice of the optimizer and subset sizes for the parameters **P, Q, R**, as the time required for the pre-processing step is a major limitation. Adapting the model to other financial instruments is also a promising extension to the current work, as it allows for the study of the model's adaptation to different market regimes and economic conditions (both micro- and macro-economic), which may reveal new trading opportunities.